TP2: Launch and Secure a Cloud Development Environment with GitHub Codespaces

1. Objectives

▪ Understand the purpose of a cloud-hosted development environment and distinguish it from a locally installed virtual machine.
•Create a Linux-based GitHub Codespace from a GitHub repository.
•Explore the remote Linux environment using basic command-line tools.
•Install and run the Nginx web server inside the Codespace.
•Expose the web server through Codespaces port forwarding and test it from a browser.
•Observe basic resource usage and understand the included Codespaces quota.
•Stop or delete the Codespace safely when the lab is finished.

2. Scenario

You are a junior DevOps engineer who needs a temporary Linux development environment without installing a virtual machine on a personal computer. You will create a GitHub Codespace, inspect its Linux environment, deploy a simple Nginx web page, test it through a forwarded port, and clean up the environment.

3. Prerequisites and safety rules

- A personal GitHub account and access to github.com.
- A modern web browser and a stable Internet connection.
- Use a personal repository or an instructor-provided repository. If creating a repository, a simple public repository is sufficient for this exercise; do not upload confidential material.
- Choose the smallest available machine type (normally 2 cores) to conserve the included monthly quota.
- Do not store passwords, API keys, private keys, personal documents, or confidential data in the repository or Codespace.
- Codespaces usage is metered. Stop the Codespace when you finish and delete it if it is no longer needed.

Part A: Create a GitHub Codespace

1. Sign in to GitHub at https://github.com.
2. Create a new repository named tp2-codespaces (or use the repository provided by the instructor). A README file may be added.
3. Open the repository and select the green Code button, then the Codespaces tab.
4. Select Create codespace on main. If prompted, choose the 2-core machine.
5. Wait for the browser-based Visual Studio Code environment to finish loading.
6. Identify the Explorer panel, integrated terminal, and Ports panel.

Record the repository name, Codespace machine type, creation date/time, and the environment status.

Part B: Explore the remote Linux environment

Open Terminal → New Terminal in the Codespace and execute the following commands:

```
whoami
hostname
uname -a
pwd
df -h
free -h
uptime
```

Observe the outputs. These commands identify the current user, host, operating system/kernel, working directory, disk space, memory, and system uptime.

Part C : Install and test Nginx

In the Codespaces terminal, run:

```
sudo apt-get update
sudo apt-get install -y nginx
sudo nginx
```

Check that Nginx is running:

```
ps aux | grep '[n]ginx'
curl -I http://localhost
```

Create a simple page:

```
echo '<h1>Hello from GitHub Codespaces - TP2</h1>' | sudo tee /var/www/html/index.html
```

In the Codespaces interface, open the Ports tab. If port 80 is not listed, select Add Port and enter 80. Open the forwarded address in a browser. Keep the port visibility Private unless the instructor explicitly asks for Public access. Confirm that the page displays the expected message.

Part D: Observe cloud environment characteristics

Answer using evidence from your own Codespace:

7. On-demand self-service: Which steps did you perform yourself to create the Codespace?
8. Broad network access: How did you access the development environment and the Nginx page?
9. Resource pooling: Which infrastructure details are managed by GitHub rather than by you?
10. Rapid provisioning: How quickly was the environment made available? What would you change if more CPU or memory were needed?
11. Measured service: Where can you inspect Codespaces compute and storage usage?

Part E: Usage and cost awareness

Open GitHub account Settings → Billing and licensing (or the current Billing page) and locate Codespaces usage. Record the included quota and the usage shown for your account. The exact menu labels may change.

For personal GitHub Free accounts, GitHub currently documents an included monthly allowance of 120 core-hours and 15 GB-month of storage. A 2-core Codespace uses 2 core-hours for each clock hour it is active, so 120 core-hours correspond to 60 hours on a 2-core machine. Check the current official documentation and your own account page because quotas and policies may change. Organization/enterprise accounts may have different arrangements.

A Codespace that is stopped does not consume active compute time, but its retained storage can continue to count toward storage usage. Delete unused Codespaces when appropriate.

Part F: Clean up

12. Save the required screenshots and notes.
13. Stop the Codespace from the Codespaces menu or Command Palette.
14. If the Codespace is no longer needed, delete it from https://github.com/codespaces.
15. Verify that the Codespace is stopped or deleted.
16. Do not delete the GitHub repository unless it was created only for this lab and the instructor asks you to remove it.

Questions

Q1. What is GitHub Codespaces? Give two differences between a Codespace and a local VMware virtual machine.

.....

.....

Q2. Which repository and machine type did you use to create your Codespace?

.....

.....

Q3. What do the commands whoami, hostname, uname -a, df -h, and free -h tell you?

.....

.....

Q4. What is Nginx? Which commands did you use to install and start it?

.....

.....

Q5. How did you access the Nginx page from your browser? What is the role of port forwarding?

.....

.....

Q6. Why should a forwarded port normally remain Private? When might Public access be justified?

.....

.....

Q7. Name two cloud characteristics you observed while using Codespaces and explain each one.

.....

.....

Q8. Where did you inspect your Codespaces usage? Explain the difference between compute usage and storage usage.

.....

.....

Q9. Why should you stop or delete an unused Codespace?

.....

.....

Submit

- Screenshot 1: GitHub repository and Codespaces environment opened in the browser. Hide personal or sensitive information.
- Screenshot 2: terminal showing the outputs of whoami, hostname, and uname -a.
- Screenshot 3: terminal showing Nginx running and the curl response.
- Screenshot 4: Nginx welcome/custom page opened through the forwarded port. Do not expose the port publicly unless instructed.
- Screenshot 5: Codespaces usage page showing quota/usage, with private account details hidden.
- Screenshot 6: Codespace stopped or deleted.